

LEAGUE OF ARROW

Taller de base de datos

Integrantes:

Jordán Arias

Ronald Garrido

Fernando Lazo

Martín Oviedo

Johanny Simken

Sofía Vergara

Profesor: Matías Calderón
Ayudante: Fernando Solís

T A B L A D E C O N T E N I D O S

Contenidos del Proyecto

- 01. Contexto y Objetivos
- 02. Avance del proyecto
- 03. Arquitectura del sistema
- 04. Logica de la base de datos
- 05. Contribuciones
- 06. Ejecucion de la aplicación

CONTEXTO Y OBJETIVOS

Problemática

Se necesita implementar en una plataforma para gestionar competencias de tiro con arco, componentes geoespaciales para:

- Manejar ubicación de arqueros y objetivos
- Validar la posición del arquero dentro del perímetro del área de competencia
- Registro de rondas y puntuaciones por fecha
- Tabla de posiciones y leaderboard histórico

Objetivos

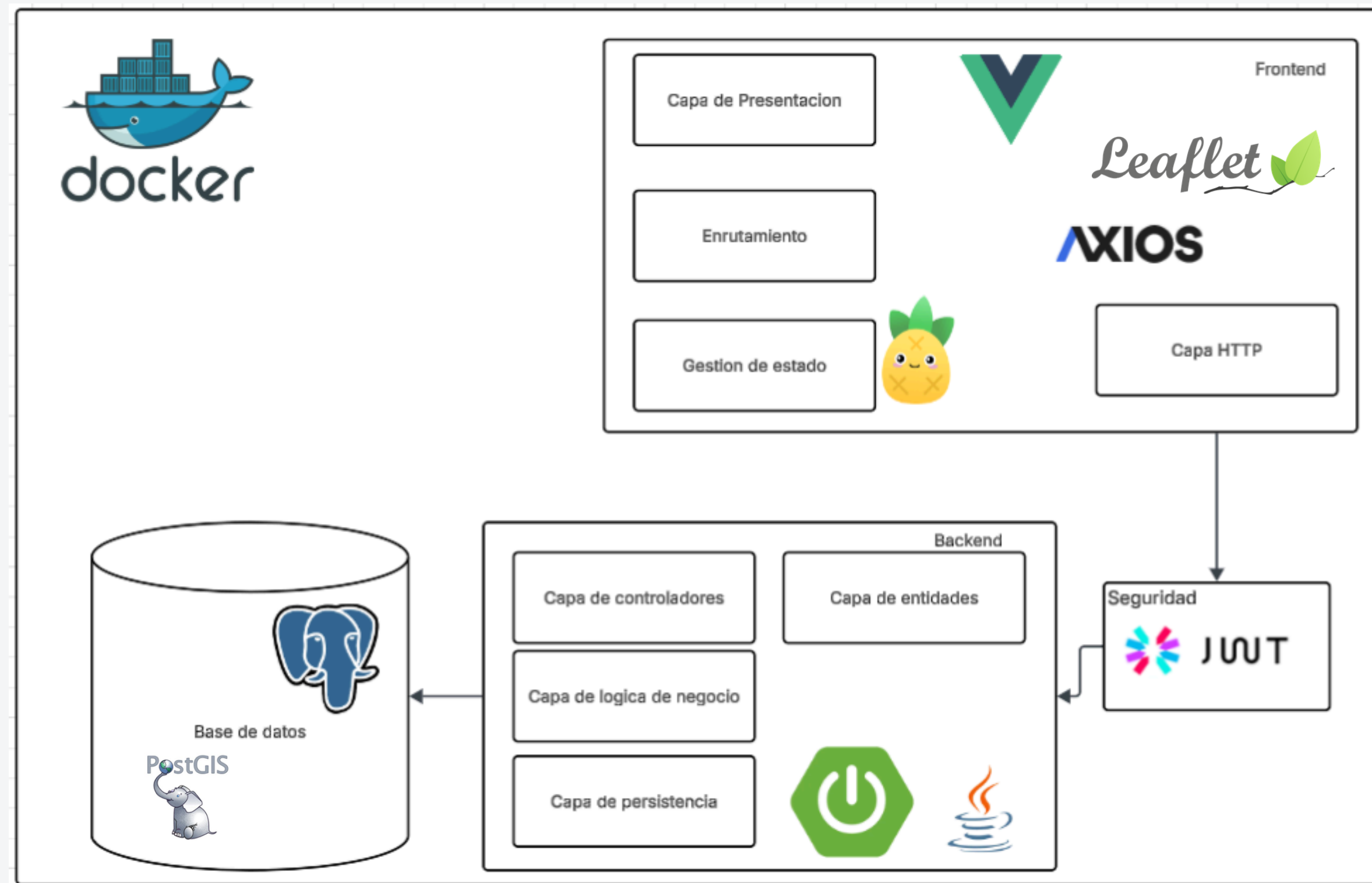
- Implementar bases de datos con uso de datos geoespaciales.
- Desarrollar una capa de servicios utilizando datos geoespaciales que garantice la integridad y seguridad de los datos.
- Gestionar la autenticación y autorización de usuarios mediante el estándar JWT.
- Documentación, guía de despliegue y uso de aplicación.

AVANCE DEL PROYECTO

- Base de Datos:** Implementación completa de postgis, triggers, indexaciones, procedimientos almacenados, vistas materializadas
- Backend:** Desarrollo de arquitectura en capas, consultas geoespaciales SQL, implementación de JWT, desarrollo de CRUD's y API's, falta una implementación de consulta SQL.
- Frontend:** Implementación funcional casi completa, con una consulta por integrar.

El proyecto está listo en un 90%

ARQUITECTURA DEL SISTEMA



LOGICA DE LA BASE DE DATOS

Componente Espacial

Ubicacion del arquero y objetivo

```
12  
13 CREATE OR REPLACE PROCEDURE registrar_puntuacion_ronda(  
14     p_tournament_id BIGINT,  
15     p_archer_id BIGINT,  
16     p_round_number INT,  
17     p_scores INTEGER[],  
18     p_lon DOUBLE PRECISION DEFAULT NULL, -- longitud GPS del arquero  
19     p_lat DOUBLE PRECISION DEFAULT NULL, -- latitud GPS del arquero  
20     p_target_id BIGINT DEFAULT NULL      -- blanco asignado  
21 )
```

Tarea PostGIS

Validacion del arquero dentro de la zona de competencia

```
86 CREATE OR REPLACE FUNCTION fn_validar_zona_competencia()
87 RETURNS TRIGGER
88 LANGUAGE plpgsql
89 AS $$
90 DECLARE
91     v_dentro BOOLEAN;
92 BEGIN
93     -- Si aún no se conoce la ubicación (ej. torneos sin componente espacial), no bloqueamos
94     IF NEW.archer_location IS NULL THEN
95         RETURN NEW;
96     END IF;
97
98     SELECT EXISTS (
99         SELECT 1
100         FROM competition_zones cz
101         WHERE cz.id_tournament = NEW.id_tournament
102             AND ST_Contains(cz.zone_polygon, NEW.archer_location)
103     ) INTO v_dentro;
104
105     IF NOT v_dentro THEN
106         RAISE EXCEPTION 'El arquero % no se encuentra dentro de la Zona de Competencia del torneo %.',
107             NEW.id_archer, NEW.id_tournament;
108     END IF;
109
110     RETURN NEW;
111 END;
112 $$;
113
114 DROP TRIGGER IF EXISTS trg_validar_zona_competencia ON rounds;
115 CREATE TRIGGER trg_validar_zona_competencia
116 BEFORE INSERT OR UPDATE OF archer_location ON rounds
117 FOR EACH ROW
118 EXECUTE FUNCTION fn_validar_zona_competencia();
```

```
127 CREATE OR REPLACE FUNCTION fn_validar_geocerca_campo()
128 RETURNS TRIGGER
129 LANGUAGE plpgsql
130 AS $$
131 DECLARE
132     v_boundary GEOMETRY;
133     v_activo BOOLEAN;
134 BEGIN
135     IF NEW.archer_location IS NULL THEN
136         RETURN NEW;
137     END IF;
138
139     SELECT f.boundary, t.is_active
140     INTO v_boundary, v_activo
141     FROM tournaments t
142     LEFT JOIN fields f ON f.id_field = t.id_field
143     WHERE t.id_tournament = NEW.id_tournament;
144
145     IF NOT v_activo THEN
146         RAISE EXCEPTION 'El torneo % no está activo: el puntaje no es válido.', NEW.id_tournament;
147     END IF;
148
149     -- Si el torneo no tiene campo oficial asociado en la tabla fields,
150     -- omitir esta validación (fn_validar_zona_competencia ya valida
151     -- que el arquero esté dentro de la competition_zone dibujada por el admin)|
152     IF v_boundary IS NULL THEN RETURN NEW; END IF;
153
154     IF NOT ST_Contains(v_boundary, NEW.archer_location) THEN
155         RAISE EXCEPTION 'Puntaje inválido: el arquero % está fuera del campo de tiro oficial.', NEW.id_archer;
156     END IF;
157
158     RETURN NEW;
159 END;
160 $$;
161
162 DROP TRIGGER IF EXISTS trg_validar_geocerca_campo ON rounds;
163 CREATE TRIGGER trg_validar_geocerca_campo
164 BEFORE INSERT OR UPDATE OF archer_location ON rounds
165 FOR EACH ROW
166 EXECUTE FUNCTION fn_validar_geocerca_campo();
```


Procedimiento Almacenado

Calcular la distancia entre la posición del arquero y el centro del blanco asignado

```
CREATE OR REPLACE FUNCTION fn_verificar_distancia_normativa(p_round_id BIGINT)
RETURNS TABLE (
    distancia_real_m NUMERIC,
    distancia_normativa_m NUMERIC,
    cumple_normativa BOOLEAN
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_archer_loc GEOMETRY;
    v_target_loc GEOMETRY;
    v_normativa NUMERIC;
    v_distancia NUMERIC;
    v_tolerancia NUMERIC := 1.0; -- tolerancia de 1 metro
BEGIN
    SELECT r.archer_location, tg.location, tg.required_distance_m
    INTO v_archer_loc, v_target_loc, v_normativa
    FROM rounds r
    JOIN targets tg ON tg.id_target = r.id_target
    WHERE r.id_round = p_round_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'La ronda % no tiene blanco o ubicación asignados.', p_round_id;
    END IF;

    v_distancia := ST_Distance(v_archer_loc::geography, v_target_loc::geography);

    RETURN QUERY
    SELECT ROUND(v_distancia, 2),
           v_normativa,
           (ABS(v_distancia - v_normativa) <= v_tolerancia);
END;
```

Validación de Seguridad:

Distancia mínima de 5 metros entre arqueros activos

```
CREATE OR REPLACE FUNCTION fn_validar_seguridad_distancia()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_conflicto RECORD;
BEGIN
    IF NEW.archer_location IS NULL THEN
        RETURN NEW;
    END IF;

    SELECT r.id_archer,
           ST_Distance(r.archer_location::geography, NEW.archer_location::geography) AS distancia
    INTO v_conflicto
    FROM rounds r
    JOIN tournaments t ON t.id_tournament = r.id_tournament
    WHERE r.id_tournament = NEW.id_tournament
           AND r.id_archer <> NEW.id_archer
           AND r.archer_location IS NOT NULL
           AND t.is_active = TRUE
           AND ST_Distance(r.archer_location::geography, NEW.archer_location::geography) < 5
    ORDER BY distancia ASC
    LIMIT 1;

    IF FOUND THEN
        RAISE EXCEPTION 'Ubicación insegura: el arquero % está a % m del arquero % (mínimo permitido: 5 m).',
            NEW.id_archer, ROUND(v_conflicto.distancia::numeric, 2), v_conflicto.id_archer;
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_validar_seguridad_distancia ON rounds;
CREATE TRIGGER trg_validar_seguridad_distancia
BEFORE INSERT OR UPDATE OF archer_location ON rounds
FOR EACH ROW
EXECUTE FUNCTION fn_validar_seguridad_distancia();
```

Análisis Espacial de Desempeño

Correlación entre la ubicación del blanco en el terreno y la precisión del arquero, considerando posibles factores ambientales

```
CREATE OR REPLACE VIEW vw_desempeno_por_condicion_ambiental AS
SELECT
    ez.zone_type          AS condicion_ambiental,
    ez.intensity          AS intensidad,
    ar.id_archer,
    ar.name               AS arquero,
    r.id_round,
    AVG(a.score)          AS precision_promedio_flecha
FROM rounds r
JOIN targets tg          ON tg.id_target = r.id_target
JOIN arrows a            ON a.id_round = r.id_round
JOIN archers ar          ON ar.id_archer = r.id_archer
JOIN environmental_zones ez ON ez.id_tournament = r.id_tournament
                        AND ST_Contains(ez.zone_polygon, tg.location)
GROUP BY ez.zone_type, ez.intensity, ar.id_archer, ar.name, r.id_round;

-- Consulta de correlación: compara el promedio de precisión
-- entre distintas condiciones ambientales (ej. viento fuerte vs sin viento)
-- y calcula el coeficiente de correlación de Pearson entre "intensidad"
-- (convertida a escala numérica) y la precisión.
SELECT
    condicion_ambiental,
    intensidad,
    COUNT(*)              AS rondas_evaluadas,
    ROUND(AVG(precision_promedio_flecha)::numeric, 3) AS promedio_precision
FROM vw_desempeno_por_condicion_ambiental
GROUP BY condicion_ambiental, intensidad
ORDER BY condicion_ambiental, intensidad;

-- Coeficiente de correlación de Pearson (usando corr() nativo de PostgreSQL)
-- entre intensidad ambiental (mapeada a 1=LEVE, 2=MODERADO, 3=FUERTE) y precisión.
SELECT
    condicion_ambiental,
    corr(
        CASE intensidad WHEN 'LEVE' THEN 1 WHEN 'MODERADO' THEN 2 WHEN 'FUERTE' THEN 3 END,
        precision_promedio_flecha
    ) AS coeficiente_correlacion
FROM vw_desempeno_por_condicion_ambiental
GROUP BY condicion_ambiental;
```

Contribuciones



EJECUCIÓN DE LA APLICACIÓN





MUCHAS
GRACIAS